

TP11 — Créer une suite de tests automatisés réseau

Formation NetOps : Automatiser la gestion de son Infrastructure Réseau — Ambient IT

Description

Ce TP formalise en véritable suite de tests ce qu'on a fait de façon ponctuelle aux TP9-TP10 : des pre-checks (avant tout déploiement — le site répond-il, la donnée est-elle cohérente) et des post-checks (après déploiement — le résultat produit est-il correct), avec un rapport consolidé par site (nombre de tests réussis/échoués) et un échec propre du playbook si un test ne passe pas.

Prérequis

- TP1 à TP10 validés

Étapes

Étape 1 — Créer sa branche

```
cd netops-project-<nom-groupe>
git checkout dev
git pull origin dev
git checkout -b <prenom>-<nom>-jour3-tp11
```

Étape 2 — Rendre device_id_prefix disponible globalement

Cette variable n'existait jusqu'ici que dans les valeurs par défaut du rôle base_config — invisible pour des tâches de test exécutées en dehors de ce rôle. On la rend disponible partout :

```
echo 'device_id_prefix: "router"' >> inventory/group_vars/all/vars.yml
cat inventory/group_vars/all/vars.yml
```

Étape 3 — Créer les pre-checks

```
mkdir -p tests/pre_checks tests/post_checks
cat > tests/pre_checks/pre_check_site.yml << 'EOF'
---
# Pre-checks : verifie l'etat AVANT tout deploiement.
```

```

- name: "PRE-CHECK : le site {{ inventory_hostname }} repond (health check)"
  block:
    - ansible.builtin.uri:
        url: "{{ api_url }}/health"
        method: GET
        return_content: yes
        register: health_probe

    - ansible.builtin.assert:
        that:
          - health_probe.json.status == "ok"
  rescue:
    - ansible.builtin.set_fact:
        test_results: "{{ (test_results | default([])) + [{'phase': 'pre', 'name': 'site_joinable', 'host': inventory_hostname, 'passed': false}] }}"
  always:
    - ansible.builtin.set_fact:
        test_results: "{{ (test_results | default([])) + [{'phase': 'pre', 'name': 'site_joinable', 'host': inventory_hostname, 'passed': true}] }}"
    when: health_probe is defined and health_probe.json.status == "ok"

- name: "PRE-CHECK : le nombre d'interfaces attendu correspond a l'API"
  block:
    - ansible.builtin.uri:
        url: "{{ api_url }}/api/devices/{{ device_id_prefix }}-{{ inventory_hostname }}-01/interfaces"
        method: GET
        headers:
          X-Api-Token: "{{ vault_api_token }}"
        return_content: yes
        register: iface_probe

    - ansible.builtin.assert:
        that:
          - iface_probe.json.interfaces | length == interfaces | length

    - ansible.builtin.set_fact:
        test_results: "{{ (test_results | default([])) + [{'phase': 'pre', 'name': 'nombre_interfaces_coherent', 'host': inventory_hostname, 'passed': true}] }}"
  rescue:
    - ansible.builtin.set_fact:
        test_results: "{{ (test_results | default([])) + [{'phase': 'pre', 'name': 'nombre_interfaces_coherent', 'host': inventory_hostname, 'passed': false}] }}"

- name: "PRE-CHECK : tous les VLAN sont dans la plage autorisee (1-4094)"
  block:
    - ansible.builtin.assert:
        that:
          - item['vlan-id'] >= 1
          - item['vlan-id'] <= 4094
        loop: "{{ interfaces }}"
        loop_control:
          label: "{{ item.name }}"

```

```

- ansible.builtin.set_fact:
    test_results: "{{ (test_results | default([])) + [{'phase': 'pre', 'name': 'vlans_dans_la_plage', 'host': inventory_hostname, 'passed': true}] }}"
rescue:
- ansible.builtin.set_fact:
    test_results: "{{ (test_results | default([])) + [{'phase': 'pre', 'name': 'vlans_dans_la_plage', 'host': inventory_hostname, 'passed': false}] }}"
EOF

```

Étape 4 — Créer les post-checks

```
cat > tests/post_checks/post_check_site.yml << 'EOF'
```

```
---
```

```
# Post-checks : verifient l'etat APRES le deployment.
```

```

- name: "POST-CHECK : le fichier de configuration a bien ete genere"
  block:
    - ansible.builtin.stat:
        path: "{{ playbook_dir }}/../generated_configs/{{ inventory_hostname }}.cfg"
        register: cfg_file

    - ansible.builtin.assert:
        that:
          - cfg_file.stat.exists

    - ansible.builtin.set_fact:
        test_results: "{{ (test_results | default([])) + [{'phase': 'post', 'name': 'fichier_config_existe', 'host': inventory_hostname, 'passed': true}] }}"
  rescue:
    - ansible.builtin.set_fact:
        test_results: "{{ (test_results | default([])) + [{'phase': 'post', 'name': 'fichier_config_existe', 'host': inventory_hostname, 'passed': false}] }}"

- name: "POST-CHECK : la configuration contient les invariants attendus"
  block:
    - ansible.builtin.assert:
        that:
          - "'ntp server' in lookup('file', playbook_dir + '/../generated_configs/' + inventory_hostname + '.cfg')"
          - "'banner motd' in lookup('file', playbook_dir + '/../generated_configs/' + inventory_hostname + '.cfg')"
          - "'hostname ' + inventory_hostname in lookup('file', playbook_dir + '/../generated_configs/' + inventory_hostname + '.cfg'"

    - ansible.builtin.set_fact:
        test_results: "{{ (test_results | default([])) + [{'phase': 'post', 'name': 'invariants_presents', 'host': inventory_hostname, 'passed': true}] }}"
  rescue:
    - ansible.builtin.set_fact:
        test_results: "{{ (test_results | default([])) + [{'phase': 'post', 'name': 'invariants_presents', 'host': inventory_hostname, 'passed': false}] }}"

- name: "POST-CHECK : pas de VLAN duplique entre les interfaces actives du site"
  vars:
    vlans_actifs: "{{ interfaces | selectattr('enabled', 'equalto', 'up') | map(attribute='vlan-id') | list }}"
  block:

```

```

- ansible.builtin.assert:
    that:
      - vlans_actifs | length == vlans_actifs | unique | length

- ansible.builtin.set_fact:
    test_results: "{{ (test_results | default([])) + [{'phase': 'post', 'name': 'pas_de_vlan_duplique', 'host': inventory_hostname, 'passed': true}] }}"
rescue:
- ansible.builtin.set_fact:
    test_results: "{{ (test_results | default([])) + [{'phase': 'post', 'name': 'pas_de_vlan_duplique', 'host': inventory_hostname, 'passed': false}] }}"
EOF

```

Étape 5 — Créer le playbook orchestrateur avec rapport final

```

cat > playbooks/run_tests.yml << 'EOF'
---
- name: Suite de tests automatisés réseau (pre-checks + post-checks)
  hosts: all
  gather_facts: no

  tasks:
    - name: Executer les pre-checks
      ansible.builtin.include_tasks: "{{ playbook_dir }}/../tests/pre_checks/pre_check_site.yml"

    - name: Executer les post-checks
      ansible.builtin.include_tasks: "{{ playbook_dir }}/../tests/post_checks/post_check_site.yml"

    - name: Afficher le rapport de tests pour ce site
      ansible.builtin.debug:
        msg: "{{ item.phase | upper }} [{{ 'OK' if item.passed else 'EHEC' }}] {{ item.name }}"
        loop: "{{ test_results }}"
        loop_control:
          label: "{{ item.name }}"

    - name: Calculer le bilan global de ce site
      ansible.builtin.set_fact:
        nb_tests: "{{ test_results | length }}"
        nb_echecs: "{{ test_results | selectattr('passed', 'equalto', false) | list | length }}"

    - name: Afficher le bilan final du site
      ansible.builtin.debug:
        msg: "BILAN {{ inventory_hostname }} : {{ nb_tests | int - nb_echecs | int }}/{{ nb_tests }} tests réussis"

    - name: Faire échouer le playbook si au moins un test a échoué sur ce site
      ansible.builtin.fail:
        msg: "{{ nb_echecs }} test(s) en échec sur {{ inventory_hostname }} : déploiement a ne pas considérer comme valide"
      when: nb_echecs | int > 0
EOF

```

Étape 6 — Exécuter la suite complète (tout doit passer)

```
cd lab && docker compose up -d --build && cd ..  
ansible-playbook --vault-password-file .vault_pass playbooks/run_tests.yml
```

Résultat attendu : BILAN <site> : 6/6 tests réussis pour les 3 sites.

Étape 7 — Provoquer volontairement l'échec d'un test

```
cp inventory/host_vars/agence2.yml /tmp/agence2_backup.yml  
cat > inventory/host_vars/agence2.yml << 'EOF'  
---  
interfaces:  
- name: GigabitEthernet0/1  
  description: LAN utilisateurs agence2  
  enabled: up  
  vlan-id: 40  
- name: GigabitEthernet0/2  
  description: Reserve  
  enabled: up  
  vlan-id: 40  
EOF
```

On active les deux interfaces avec le même VLAN — une vraie erreur de configuration réseau (conflit de VLAN sur un même site).

```
ansible-playbook --vault-password-file .vault_pass playbooks/run_tests.yml --limit agence2
```

Résultat attendu : POST [ECHEC] pas_de_vlan_duplique, BILAN agence2 : 5/6 tests réussis, et le playbook échoue proprement pour ce site (les 5 autres tests restent verts).

Étape 8 — Restaurer l'état correct

```
cp /tmp/agence2_backup.yml inventory/host_vars/agence2.yml  
ansible-playbook --vault-password-file .vault_pass playbooks/run_tests.yml --limit agence2
```

Doit repasser à 6/6 tests réussis.

Étape 9 — Committer et pousser

```
git add tests/ playbooks/run_tests.yml inventory/group_vars/  
git commit -m "TP11 : suite de tests automatisés reseau (pre-checks + post-checks)"  
git push -u origin <prenom>-<nom>-jour3-tp11
```

Ouvrir une pull request <prenom>-<nom>-jour3-tp11 → dev.

Comprendre le TP

Pre-checks vs post-checks : les pre-checks valident l'environnement avant d'agir (le site répond-il, la donnée qu'on s'apprête à utiliser est-elle valide) — leur but est

d'éviter de commencer un déploiement voué à l'échec. Les post-checks valident le résultat une fois l'action faite — leur but est de ne jamais considérer un déploiement comme terminé tant qu'on n'a pas vérifié qu'il a produit ce qu'on attendait.

Le pattern `test_results` : chaque test suit le même schéma block (la vérification) / rescue (si elle échoue, on enregistre `passed: false`) / suite normale (si elle réussit, `passed: true`) — cohérent avec la gestion d'exceptions du TP9, mais appliqué ici à des tests nommés plutôt qu'à des interfaces. La liste `test_results` s'enrichit au fil du playbook, permettant un rapport consolidé à la fin plutôt qu'un arrêt au premier échec.

Le test `pas_de_vlan_duplique` illustre une règle métier réseau réelle : deux interfaces actives sur le même VLAN sur un même site peuvent créer des conflits de commutation — un test automatisé l'attrape immédiatement, là où un déploiement manuel ne l'aurait peut-être remarqué qu'après un incident.

Validation

- ☐ `playbooks/run_tests.yml` s'exécute et affiche 6/6 tests réussis pour les 3 sites en temps normal
- ☐ Un doublon de VLAN volontaire fait échouer précisément le test `pas_de_vlan_duplique`, sans casser les autres
- ☐ Le playbook échoue proprement (`ansible.builtin.fail`) pour le site en défaut, avec un message clair
- ☐ Une pull request `<prenom>-<nom>-jour3-tp11` → dev est ouverte